

CRR-Binomial-Pricing-Build-Sequence

PLAN2.md -- Layer 1 (Done) and Layer 2 (New) Implementation Guide

github.com/dantzlerdc/CS495-Deep-Scholar

Color Key

Layer 1 -- Already Built

Layer 2 -- New (to build)

Overview

PLAN2.md defines a two-layer pipeline. Layer 1 (the CRR pricing engine) is fully implemented -- all six stages run today via 'make run'. Layer 2 adds the components required by Project 6 that are absent from the original pipeline: an independent probability estimator, a crowd bias/regime detector, a microstructure cost model, probability calibration, a trade policy engine, and a walk-forward backtest runner. Build Layer 2 in the step order below -- each module depends on the one before it.

Recommended Build Order

Step	Module	Status	Purpose
0	make run	Done	Confirm Layer 1 pipeline executes end-to-end
1	market_data.py	New	Layer 2 data feed: IV surface, volume, OI, spread
2	p_estimator.py	New	ML independent probability model (core gap)
3	bias_detector.py	New	Regime classifier: normal vs. herding crowd
4	micro_cost.py	New	Fee / spread / slippage cost filter
5	calibration.py	New	Brier score + calibration curve validation
6	policy.py	New	Combined regime-conditional trade decision
7	backtest.py	New	Walk-forward historical backtest + dashboard plots

Prerequisites Before Starting Layer 2

Historical option chain data

p_estimator.py needs labeled ITM/OTM outcomes. Use the Kaggle AAPL dataset (kylegraupe/aapl-options-data-2016-2020) as a proxy if live AMD history is unavailable. Document the substitution in the report.

scikit-learn and XGBoost installed

Run: `.venv/bin/python3 -m pip install scikit-learn xgboost lightgbm`

Layer 1 passing cleanly

The backtest uses CRR pricing as a comparison benchmark. tree.py must produce V_model within 0.005% of V_market before Layer 2 begins.

Step 0 make run (Layer 1 verification)

Confirm all six CRR pipeline stages execute before starting Layer 2.

Status: Layer 1 -- Already Built

What it does

Runs the full six-stage CRR pipeline end-to-end: data ingestion (data.py), CRR pricing (tree.py), Greeks (greeks.py), mispricing edge (edge.py), Kelly sizing (kelly.py), and Monte Carlo simulation (simulation.py). All outputs are written to project/outputs/ as CSV files and PNG plots. If this command completes without errors, Layer 1 is your foundation and Layer 2 build can begin.

Depends on

config.yaml -- AMD option chain parameters (S, K, r, IV, T, N)

.venv active -- all Layer 1 dependencies installed via 'make setup'

Note: Run from the repo root. If it fails, fix Layer 1 before touching Layer 2.

```
make setup    # first-time only: create .venv and install dependencies
make run      # execute all six pipeline stages
```

Step 1 market_data.py

Layer 2 data feed -- IV surface, volume, open interest, bid-ask spread.

Status: Layer 2 -- New

What it does

Fetches the additional market data that Layer 2 needs but Layer 1 does not collect: the full implied volatility surface across multiple strikes and expiries, option volume, open interest, and bid-ask spread. These are the raw inputs for the probability estimator (Step 2), the regime detector (Step 3), and the microstructure cost model (Step 4). All data is appended to the existing pandas DataFrame produced by data.py.

Depends on

data.py -- existing Layer 1 data module (market_data.py extends it)

yfinance -- already installed; no new dependencies needed for this step

Note: Everything else in Layer 2 depends on this module. Build and validate it before writing any other Layer 2 code.

Key Functions to Implement

```
fetch_iv_surface(ticker, expiries)    -- returns IV grid (strike x expiry)
fetch_volume_oi(ticker, expiry)      -- returns volume and open interest per strike
fetch_bid_ask_spread(ticker, expiry) -- returns mid-price and half-spread per strike
build_layer2_df(cfg, base_df)        -- merges all Layer 2 features into one DataFrame
```

Step 2 p_estimator.py

Independent ML probability model -- the core gap from PLAN.md.

Status: Layer 2 -- New

What it does

Trains a gradient boosting classifier (XGBoost or LightGBM) on historical option outcomes. Each training example is a historical option contract; the label is 1 (expired ITM) or 0 (expired OTM). Features include moneyness (S/K), days-to-expiry, the RV-IV spread, volume/OI ratio, and bid-ask spread. The model outputs `p_independent` -- a probability that does NOT use the market price as an input. This is what makes it a genuine independent predictor. After training, Platt scaling or isotonic regression is applied so that `p_independent=0.70` actually means the option expires ITM 70% of the time. The calibrated output feeds directly into `edge_independent = p_independent - q_market`.

Depends on

`market_data.py` -- for feature data (Step 1 must be complete)

Historical labeled data -- Kaggle AAPL dataset or AMD chain snapshots

scikit-learn, XGBoost/LightGBM -- install before this step

Note: This is the most important new module. The original CRR pipeline has edge ~ 0% because IV is solved FROM the market price. p_estimator.py breaks that circularity by forecasting p from features, not from market IV.

Key Functions to Implement

```
build_features(df)           -- engineers feature matrix from Layer 2 DataFrame
train_p_model(X_train, y_train) -- fits gradient boosting; returns model
calibrate_p(model, X_cal, y_cal) -- Platt scaling / isotonic regression
predict_p_independent(model, X) -- returns calibrated p_independent per ticket
compute_edge_independent(p, q_market) -- returns edge_independent = p - q_market
```

Step 3 bias_detector.py*Crowd regime classifier -- normal vs. herding market conditions.***Status:** Layer 2 -- New**What it does**

Computes behavioral bias indicators from the IV surface and volume data, then classifies each trading day into one of two regimes. Normal regime: the crowd is approximately rational; IV tracks realized volatility closely; trade any edge above the 2% minimum threshold. Herding regime: the crowd is systematically biased; IV is detached from RV; vol premium is large and persistent; raise the minimum edge threshold to 8% and trade only high-conviction signals. Three primary indicators are used: (1) RV-IV spread -- the difference between implied volatility and 30-day realized volatility; (2) IV momentum -- the 3-day and 5-day rate of change in at-the-money IV; (3) volume/OI ratio -- abnormal option flow relative to open interest as a herding proxy.

Depends on

market_data.py -- IV surface, volume, OI (Step 1)

p_estimator.py -- p_independent used to confirm mispricing persistence (Step 2)

*Note: Regime thresholds (5% / 10% RV-IV spread) should be validated on historical data.***Regime Logic**

```
RV_IV_spread    = IV_implied - RV_30d
Normal regime   : abs(RV_IV_spread) < 5%  -- min edge threshold = 2%
Herding regime  : RV_IV_spread > 10%      -- min edge threshold = 8%
```

Step 4 micro_cost.py*Fee, spread, and slippage cost model -- edge net of transaction costs.***Status:** Layer 2 -- New**What it does**

Calculates the true net edge after subtracting all transaction costs from the raw edge signal. Costs modeled: bid-ask spread (half-spread paid on entry and exit), brokerage fee (\$0.65 per contract at Fidelity), and slippage (price impact when order size is large relative to daily volume). The key output is `edge_net` -- the edge that actually survives into your P&L after costs. The policy in Step 6 gates trade entry on `edge_net > min_edge_threshold`. An additional VaR constraint rejects any trade where the Kelly-sized position would exceed 5% of capital at risk. A drawdown circuit breaker halts all trading if cumulative losses exceed 15% of starting capital.

Depends on

`market_data.py` -- bid-ask spread data per strike (Step 1)

`edge.py` -- raw CRR edge (Layer 1, already built)

`p_estimator.py` -- `edge_independent` (Step 2)

Note: Can be built in parallel with Step 3 -- does not depend on `bias_detector.py`.

Net Edge Formula

```
cost = (0.5 * bid_ask_spread + fee_per_share + slippage) / V_market
edge_net = edge_independent - cost
Trade only when: edge_net > min_edge_threshold (regime-conditional)
```

Step 5 calibration.py

Brier score and calibration curve -- validate p_independent accuracy.

Status: Layer 2 -- New

What it does

Evaluates whether p_independent from Step 2 is trustworthy. A model that outputs $p=0.70$ should be correct 70% of the time. Calibration.py measures this using two tools. The Brier score is the mean squared error between predicted probabilities and actual outcomes (lower is better; a naive baseline of always predicting 0.50 scores 0.25). The reliability diagram bins predictions into deciles and plots mean predicted probability versus actual outcome frequency -- a well-calibrated model falls on the diagonal. The Expected Calibration Error (ECE) summarizes the reliability diagram as a single number. All three metrics are reported in the research report Section 6. The calibration curve PNG is saved to project/outputs/.

Depends on

p_estimator.py -- trained and calibrated model + held-out test set (Step 2)

Note: The Brier score baseline is 0.25 (naive 50/50 prediction). The model must beat this to demonstrate any predictive value.

Key Metrics

```
Brier score = mean((p_pred - outcome)^2)    # baseline = 0.25; lower is better
ECE          = mean(|mean_pred - mean_actual|) per probability bin
Output: calibration_curve.png -> project/outputs/
```

Step 6 policy.py

Regime-conditional trade decision engine -- where all layers connect.

Status: Layer 2 -- New

What it does

Combines the outputs of all prior Layer 2 modules into a single trade decision per ticket. For each option contract the policy asks: (1) Is the regime normal or herding? (sets the edge threshold). (2) Does edge_net exceed the regime-appropriate threshold? (3) Does the Kelly-sized position pass the VaR and drawdown constraints? If all three conditions are met, the policy emits a TRADE signal with the full/half/quarter-Kelly dollar position. Otherwise it emits NO TRADE. This is the direct analog of the Kalshi-style trading policy described in Project 6 Section 2.

Depends on

bias_detector.py -- regime label (Step 3)

micro_cost.py -- edge_net (Step 4)

kelly.py -- Kelly fraction (Layer 1, already built)

Note: This is the capstone of Layer 2 -- all prior steps feed into it.

Decision Logic

```

min_edge = 0.02 if regime == 'normal' else 0.08    # regime-conditional
if edge_net > min_edge and kelly_position <= var_limit:
    signal = TRADE    # with dollar_full / dollar_half / dollar_quarter
else:
    signal = NO_TRADE

```

Step 7 backtest.py

Walk-forward historical backtest runner and experiment dashboard.

Status: Layer 2 -- New

What it does

Applies policy.py to historical option chain data in a walk-forward loop: train on the first N months, test on the next month, then roll the window forward. For each test period, records the trade signal, the Kelly position, and the realized P&L (did the option expire ITM or OTM?). Aggregates results across all windows and computes P&L distribution, annualized Sharpe ratio, maximum drawdown, hit rate, and Brier score. Saves three plots to project/outputs/: pnl_curve.png (cumulative P&L over the backtest), regime_analysis.png (RV-IV spread and regime state over time), and a summary comparison of full/half/quarter-Kelly equity curves.

Depends on

policy.py -- trade decision for each contract (Step 6)

calibration.py -- Brier score for backtest window (Step 5)

Historical labeled data -- same dataset used to train p_estimator.py

Note: Build this last. It is the experiment you report in the research report Section 6 (backtest results) and present on Slide 5 of the final presentation.

Key Output Metrics

P&L distribution	-- histogram of trade-level profits and losses
Sharpe ratio	-- annualized (P&L mean / P&L std * sqrt(252))
Max drawdown	-- largest peak-to-trough cumulative loss
Hit rate	-- fraction of trades where model direction was correct
Brier score	-- probability calibration accuracy on backtest outcomes
Outputs: pnl_curve.png, regime_analysis.png -> project/outputs/	

Prerequisites Checklist Before Starting Layer 2

Historical option chain data

`p_estimator.py` requires labeled ITM/OTM outcomes at expiry. Use the Kaggle AAPL options dataset (kylegraupe/aapl-options-data-2016-2020) as a structural proxy if AMD chain history is unavailable. Document the data substitution explicitly in the research report limitations section.

scikit-learn and XGBoost installed

Run the command below from the repo root before starting Step 2.

Layer 1 passing cleanly

The walk-forward backtest (Step 7) uses CRR pricing as a comparison benchmark. `tree.py` must produce `V_model` within 0.005% of `V_market`. Run 'make run' and confirm `outputs/model_vs_market.png` shows near-zero error before starting Layer 2.

Project/outputs directory writable

Steps 5 and 7 write PNG files to `project/outputs/`. Verify the directory exists and is writable: `ls project/outputs/`

Install Layer 2 Dependencies

```
.venv/bin/python3 -m pip install scikit-learn xgboost lightgbm
```

Build Command Quick Reference

```
# Step 0 -- verify Layer 1
make run
```

```
# Steps 1-7 -- Layer 2 (run from repo root, in order)
.venv/bin/python3 project/market_data.py
.venv/bin/python3 project/p_estimator.py
.venv/bin/python3 project/bias_detector.py
.venv/bin/python3 project/micro_cost.py
.venv/bin/python3 project/calibration.py
.venv/bin/python3 project/policy.py
.venv/bin/python3 project/backtest.py
```

```
# Rebuild PDF deliverables
.venv/bin/python3 make_crr_pdf.py
.venv/bin/python3 make_pipeline_components_pdf.py
.venv/bin/python3 make_build_sequence_pdf.py
```

```
# Rebuild HTML animation
.venv/bin/python3 crr_binomial_pricing_amd_html.py
```

Layer 2 -- All Modules Built: Complete Run Sequence

All seven Layer 2 modules are written and ready to run. Execute the steps below in order from the repo root. Each step depends on the one before it. Steps 2 and 7 take the longest (model training and walk-forward backtest).

Step	Command	Output file(s)	What it produces
0	make run	outputs/*.csv, *.png	Verify Layer 1: CRR pricing, Greeks, edge, Kelly, Monte Carlo
1	project/market_data.py	(DataFrame in memory)	Load AAPL dataset, compute RV, label ITM/OTM, engineer fe
2	project/p_estimator.py	outputs/call_model.pkl	Train and calibrate gradient boosting probability models
3	project/bias_detector.py	outputs/regime_analysis.png	Classify normal vs herding regime; plot RV-IV spread over tim
4	project/micro_cost.py	(console report)	Verify net edge after spread, fee, slippage on AMD tickets
5	project/calibration.py	outputs/calibration_curve.png	Brier score + reliability diagram for call and put models
6	project/policy.py	(console report)	Regime-conditional trade decision for each AMD ticket
7	project/backtest.py	outputs/pnl_curve.png	Walk-forward backtest: P&L, Sharpe, drawdown, hit rate

Full Run -- Copy and Paste

```
# Layer 1 (already built)
make run

# Layer 2 -- run in order from repo root
.venv/bin/python3 project/market_data.py
.venv/bin/python3 project/p_estimator.py
.venv/bin/python3 project/bias_detector.py
.venv/bin/python3 project/micro_cost.py
.venv/bin/python3 project/calibration.py
.venv/bin/python3 project/policy.py
.venv/bin/python3 project/backtest.py
```

Expected Output Files After Full Run

File	Description
outputs/call_model.pkl	Calibrated call probability model (XGBoost + Platt scaling)
outputs/put_model.pkl	Calibrated put probability model (XGBoost + Platt scaling)
outputs/regime_analysis.png	ATM IV vs RV30, RV-IV spread, and regime state over time
outputs/calibration_curve.png	Reliability diagram + Brier score for call and put models
outputs/pnl_curve.png	Cumulative P&L and trade P&L distribution (walk-forward backtest)
outputs/backtest_trades.csv	Full trade log: date, regime, edge, outcome, P&L per trade
outputs/model_vs_market.png	Layer 1: CRR V_model vs V_market for all four AMD tickets
outputs/convergence.png	Layer 1: CRR price convergence as N increases
outputs/kelly_fractions.png	Layer 1: full / half / quarter-Kelly dollar positions
outputs/early_exercise_boundary.png	Layer 1: AMD put early exercise boundary S* over time